Python Competitive Programming Reference

Compact team reference — algorithms, mathematics, graphs, and data structures

Python Syntax, Formatting & Fast I/O

```
# Binary string -> integer
n = int(s, 2)

# Integer -> zero-padded binary string
b = format(x, f"0{width}b")

# Float formatting
print("%.2f" % value)
print(f"{answer:.2f}")

# Read entire input as integers
import sys
data = list(map(
    int, sys.stdin.buffer.read().split()
))

# Read entire input as strings
words = sys.stdin.buffer.read().split()
# bytes -> str only when needed: token.decode()
```

Sorting a `defaultdict`

```
from collections import defaultdict

# By key
D = defaultdict(d.default_factory,
                sorted(d.items()))

# By value
D = defaultdict(d.default_factory,
                sorted(d.items(),
                       key=lambda kv: kv[1]))
```

Common Contest Terminology

Let $A = [1, 2, 3, 4]$ and $S =$ `abcd`.

Subarray: Contiguous elements of an array, e.g. `[2,3]` from A.

Subsequence: Keep original relative order; skipping is allowed, e.g. `[1,3,4]` from A.

Subset: Choose elements without caring about order, e.g. $\{1, 4\}$ from A.

Substring: Contiguous characters of a string, e.g. `"bc"` from S.

Remember: Every subarray is a subsequence. The whole object qualifies; whether empty selections count depends on the problem statement.

Modular Arithmetic

For modulus M:

$$(a + b) \bmod M = ((a \bmod M) + (b \bmod M)) \bmod M$$

$$(a - b) \bmod M = ((a \bmod M) - (b \bmod M) + M) \bmod M$$

$$(ab) \bmod M = ((a \bmod M)(b \bmod M)) \bmod M$$

In Python, `a % M` is already non-negative for positive M.

Sequences & Series

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

$$\sum_{k=0}^{n-1} (a + kd) = \frac{n(2a + (n-1)d)}{2}$$

$$\sum_{i=1}^{n} (2i - 1) = n^2$$

$$\sum_{i=1}^{n} 2i = n(n+1)$$

Arithmetic progression squares:

$$\sum_{k=0}^{n-1} (a + kd)^2 = na^2 + ad\, n(n-1) + d^2 \frac{n(n-1)(2n-1)}{6}$$

Arithmetic progression cubes:

$$\sum_{k=0}^{n-1} (a + kd)^3 = na^3 + 3a^2 d \frac{n(n-1)}{2} + 3ad^2 \frac{n(n-1)(2n-1)}{6} + d^3 \left(\frac{n(n-1)}{2}\right)^2$$

Geometric progression ($r \neq 1$):

$$\sum_{k=0}^{n-1} ar^k = a \frac{r^n - 1}{r - 1}$$

For $r = 1$, the sum is an.

Alternating Sum

```
# 1 - 2 + 3 - 4 + ... +/- n
alt_sum = -n // 2 if n % 2 == 0 else (n + 1) // 2

# 1^k - 2^k + 3^k - ... +/- n^k
alt_pow_sum = sum((1 if i & 1 else -1) * i**k
                  for i in range(1, n + 1))
```

Binary Search on Answer

First True / Minimum Valid: FFFFFTTTTT

```
def check(x):
    return ...

l, r = ..., ...
ans = -1
while l <= r:
    m = (l + r) // 2
    if check(m):
        ans = m
        r = m - 1
```

```
    else:
        l = m + 1
print(ans)
```

Last True / Maximum Valid: TTTTTFFFFF

```
def check(x):
    return ...

l, r = ..., ...
ans = -1
while l <= r:
    m = (l + r) // 2
    if check(m):
        ans = m
        l = m + 1
    else:
        r = m - 1
print(ans)
```

Requirement: `check(x)` must be monotonic over the search interval.

Array Techniques

Prefix Sum — Static Range Sum

0-based, inclusive $[l, r]$. Build $O(n)$, each query $O(1)$; no point updates.

```
a = [2, 4, 1, 5, 3]
pref = [0]
for x in a:
    pref.append(pref[-1] + x)

def range_sum(l, r):
    return pref[r + 1] - pref[l]

# range_sum(1, 3) == 10
```

Difference Array — Offline Range Add

0-based, inclusive $[l, r]$. Each update $O(1)$, all final values $O(n)$. Apply updates before reading results.

```
a = [1, 2, 3, 4, 5]
n = len(a)
diff = [0] * (n + 1)

def range_add(l, r, x):
    diff[l] += x
    diff[r + 1] -= x

range_add(1, 3, 5)
range_add(2, 4, 2)
cur = 0
for i in range(n):
    cur += diff[i]
    a[i] += cur
# a == [1, 7, 10, 11, 7]
```

Two Pointers — Pair Sum

Sorted input required. $O(n)$ scan after sorting ($O(n \log n)$ if unsorted). Indices refer to the sorted array.

```
def two_sum_sorted(a, target):
    l, r = 0, len(a) - 1
    while l < r:
        s = a[l] + a[r]
        if s == target:
            return l, r
        if s < target:
            l += 1
        else:
            r -= 1
    return None
# two_sum_sorted([1,2,4,7,11],9) == (1,3)
```

Sliding Window — Fixed Size

Maximum sum of a contiguous length-k subarray, $O(n)$, including negative values; $1 \le k \le n$.

```
def max_k_window(a, k):
    window = sum(a[:k])
    best = window
    for r in range(k, len(a)):
        window += a[r] - a[r - k]
        best = max(best, window)
    return best

# max_k_window([2,-1,3,4,-2],3) == 6
```

Kadane — Maximum Subarray Sum

Maximum sum of a *non-empty contiguous* subarray. $O(n)$ time, $O(1)$ extra space; works for all-negative arrays.

```
def kadane(a):
    cur = best = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best

# kadane([-2,1,-3,4,-1,2,1,-5,4]) == 6
```

Bit Manipulation Basics & XOR

Basic Bit Operations

Assume $x \ge 0$; bit position k is 0-based from the right.

```
x & 1                    # Odd (1) / even (0)
(x >> k) & 1             # Check bit k
x | (1 << k)             # Set bit k
x & ~(1 << k)            # Clear bit k
x ^ (1 << k)             # Toggle bit k
x.bit_count()            # Count 1-bits
x.bit_length()           # Number of bits
x & -x                   # Lowest set bit
x & (x - 1)              # Clear lowest set bit
x > 0 and (x & (x-1)) == 0  # Power of 2
```

In Python, `~x` is negative for non-negative x; mask off unwanted higher bits when using NOT.

XOR Properties and Range XOR

```
# a ^ a = 0; a ^ 0 = a; a ^ b ^ a = b

# XOR of integers from 0 through n
def xor_upto(n):
    if n < 0:
        return 0
    return (n, 1, n + 1, 0)[n % 4]

# XOR of integers in [l, r], 0 <= l <= r
def xor_range(l, r):
    return xor_upto(r) ^ xor_upto(l - 1)

# Exactly one value occurs once, all others twice
a = [4, 1, 2, 1, 2]
ans = 0
for x in a:
    ans ^= x
# ans == 4
```

Number Theory

GCD / LCM

```
from math import gcd
lcm = a // gcd(a, b) * b
```

```
# Euclid; useful when b itself is supplied via b % a
# without constructing a huge decimal integer.
def gcd_euclid(a, b):
    while a:
        a, b = b % a, a
    return b
```

Prime Factorization by Trial Division

```
from collections import defaultdict
from math import isqrt

def factors(n):
    f = defaultdict(int)
    while n % 2 == 0:
        f[2] += 1
        n //= 2
    p = 3
    while p * p <= n:
        while n % p == 0:
            f[p] += 1
            n //= p
        p += 2
    if n > 1:
        f[n] += 1
    return f
```

If $n = \prod p_i^{e_i}$:

$$\tau(n) = \prod (e_i + 1), \qquad \sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}.$$

```
def num_divisors(fact):
    ans = 1
    for e in fact.values():
        ans *= e + 1
    return ans
```

Divisor Count / Sum for Every $1 \ldots N$

```
N = 10**5
divcnt = [0] * (N + 1)
for i in range(1, N + 1):
    for j in range(i, N + 1, i):
        divcnt[j] += 1

# Sum of divisors instead:
# divsum[j] += i
```

Ordinary Sieve of Eratosthenes

```
def sieve(N):
    is_prime = bytearray(b'\x01') * (N + 1)
    if N >= 0: is_prime[0] = 0
    if N >= 1: is_prime[1] = 0
    for p in range(2, int(N**0.5) + 1):
        if is_prime[p]:
            start = p * p
            is_prime[start:N+1:p] = b'\x00' * (
                (N - start) // p + 1)
    return is_prime
```

Odd-Only Bytearray Sieve

Index i represents the odd number $2i + 1$.

```
from math import isqrt

def sieve_bytearray(N):
    size = (N + 1) // 2
    s = bytearray(b'\x01') * size
    if size:
        s[0] = 0                    # 1 is not prime

    lim = (isqrt(N) - 1) // 2
    for i in range(1, lim + 1):
        if s[i]:
            p = 2 * i + 1
            start = (p * p) // 2
            s[start::p] = b'\x00' * (
                (size - start - 1) // p + 1)
    return s

N = 10**6 + 7
a = sieve_bytearray(N)

def is_prime_fast(x):
    if x == 2:
        return True
    if x < 2 or x % 2 == 0 or x > N:
        return False
    return bool(a[x // 2])
```

Compact Prime List from Odd Sieve

```
from array import array

primes = array('I', [2])
for i in range(1, len(a)):
    if a[i]:
        primes.append(2 * i + 1)
```

Smallest Prime Factor (SPF) — Many Queries

Preprocess once; each factorization takes $O(\log n)$ factors with multiplicity.

```
N = 10**6 + 9
spf = list(range(N + 1))

for i in range(2, int(N**0.5) + 1):
    if spf[i] == i:
        for j in range(i * i, N + 1, i):
            if spf[j] == j:
                spf[j] = i

def factorize(n):
    res = []
    while n > 1:
        p = spf[n]
        res.append(p)
        n //= p
    return res
```

All Divisors of One Number

```
def all_divisors(n):
    d = []
    i = 1
    while i * i <= n:
        if n % i == 0:
            d.append(i)
            if i != n // i:
                d.append(n // i)
        i += 1
    return d              # sort(d) if order matters
```

Goldbach Pair for an Even Integer

```
def goldbach(n, is_prime):
    if n < 4 or n % 2:
        return None
    for p in range(2, n // 2 + 1):
        if is_prime[p] and is_prime[n - p]:
            return p, n - p
    return None
```

Binary Exponentiation

```
def bin_exp(a, b, mod=None):
    res = 1
    while b:
```

```
        if b & 1:
            res = (res * a if mod is None
                   else res * a % mod)
        a = (a * a if mod is None
             else a * a % mod)
        b >>= 1
    return res

# Python built-in modular exponentiation:
# pow(a, b, MOD)
```

Deterministic Miller–Rabin for 64-bit Integers

```
def is_prime_mr(n):
    if n < 2:
        return False
    small = (2, 3, 5, 7, 11, 13, 17,
             19, 23, 29, 31, 37)
    for p in small:
        if n % p == 0:
            return n == p

    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1

    for a in (2, 325, 9375, 28178, 450775,
              9780504, 1795265022):
        if a % n == 0:
            continue
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True
```

Factorials, Inverse Factorials & nCr

Requires prime `MOD` and $n <$ MOD for Fermat inverse.

```
MOD = 10**9 + 7
MAXN = 2000

fact = [1] * (MAXN + 1)
for i in range(1, MAXN + 1):
    fact[i] = fact[i - 1] * i % MOD

inv_fact = [1] * (MAXN + 1)
inv_fact[MAXN] = pow(fact[MAXN], MOD - 2, MOD)
for i in range(MAXN, 0, -1):
    inv_fact[i - 1] = inv_fact[i] * i % MOD

def nCr(n, r):
    if r < 0 or r > n:
        return 0
    return (fact[n] * inv_fact[r] % MOD
            * inv_fact[n-r] % MOD)
```

Geometry

Triangle Area — Heron's Formula

For sides a, b, c and $s = (a + b + c)/2$:

$$K = \sqrt{s(s-a)(s-b)(s-c)}.$$

Law of Cosines

For triangle ABC, side AB is opposite angle C:

```
from math import acos, degrees
C = degrees(acos((BC*BC + CA*CA - AB*AB)
                 / (2 * BC * CA)))
```

Clamp the cosine to $[-1, 1]$ if floating-point error is possible.

Polygon Area — Shoelace Formula

```
def polygon_area2(points):
    # Returns twice the area.
    # Exact for integer coordinates.
    area2 = 0
    n = len(points)
    for i in range(n):
        x1, y1 = points[i]
        x2, y2 = points[(i + 1) % n]
        area2 += x1 * y2 - y1 * x2
    return abs(area2)

# area = polygon_area2(points) / 2
```

Graph Algorithms

BFS — Unweighted Shortest Paths

1-indexed adjacency list: `graph[u] = [v,...]`. $O(V+E)$ time. `dist[v] = -1` means unreachable; `par` works with `restore_path` below.

```
from collections import deque

def bfs(n, graph, src=1):
    dist = [-1] * (n + 1)
    par = [-1] * (n + 1)
    dist[src] = 0
    q = deque([src])
    while q:
        u = q.popleft()
        for v in graph[u]:
            if dist[v] != -1:
                continue
            dist[v] = dist[u] + 1
            par[v] = u
            q.append(v)
    return dist, par
```

For non-negative weighted edges, use Dijkstra instead.

Bipartite Graph — BFS 2-Coloring

Undirected graph, 1-based adjacency list. $O(V+E)$ time and $O(V)$ space. Colors: `-1` = unvisited; `0` and `1` = groups.

```
from collections import deque

def is_bipartite(n, graph):
    color = [-1] * (n + 1)

    for start in range(1, n + 1):
        if color[start] != -1:
            continue
        color[start] = 0
        q = deque([start])

        while q:
            u = q.popleft()
            for v in graph[u]:
                if color[v] == -1:
                    color[v] = 1 - color[u]
                    q.append(v)
                elif color[v] == color[u]:
                    return False
    return True
```

Bipartite iff no odd-length cycle. Every tree is bipartite. Checks disconnected graphs and rejects self-loops.

Iterative DFS with Traversal Depth

For a tree this is depth from `start`; in a general graph it is DFS-tree depth.

```
def dfs_max_depth(start, graph):
    seen = {start}
    stack = [(start, 1)]
    best = 0
    while stack:
        u, dep = stack.pop()
        best = max(best, dep)
        for v in graph[u]:
            if v not in seen:
                seen.add(v)
                stack.append((v, dep + 1))
    return best
```

Binary Tree Inorder Traversal

```
def inorder(tree, root):
    ans, st = [], []
    cur = root
    while cur is not None or st:
        while cur is not None:
            st.append(cur)
            cur = tree.get(cur, (None, None))[0]
        cur = st.pop()
        ans.append(cur)
        cur = tree.get(cur, (None, None))[1]
    return ans
```

N-ary Tree Preorder & Postorder

```
def dfs_orders(tree, root):
    pre, post = [], []
    st = [(root, 0)]
    while st:
        u, i = st.pop()
        if i == 0:
            pre.append(u)
        ch = tree.get(u, [])
        if i < len(ch):
            st.append((u, i + 1))
            st.append((ch[i], 0))
        else:
            post.append(u)
    return pre, post
```

Dijkstra — Non-negative Edge Weights

Adjacency list: `graph[u] = [(v,w), ...]`.

```
import heapq

def dijkstra(n, graph, src=1):
    INF = float('inf')
    dist = [INF] * (n + 1)
    par = [-1] * (n + 1)
    dist[src] = 0
    pq = [(0, src)]

    while pq:
        du, u = heapq.heappop(pq)
        if du != dist[u]:
            continue
        for v, w in graph[u]:
            nd = du + w
            if nd < dist[v]:
                dist[v] = nd
                par[v] = u
                heapq.heappush(pq, (nd, v))
    return dist, par

def restore_path(par, src, target):
    path = []
    u = target
    while u != -1:
        path.append(u)
        if u == src:
            return path[::-1]
        u = par[u]
    return []
```

Topological Sort — Kahn's Algorithm

```
from collections import deque

def topo_sort(graph, n):
    indeg = [0] * (n + 1)
    for u in range(1, n + 1):
        for v in graph[u]:
            indeg[v] += 1

    q = deque(i for i in range(1, n + 1)
              if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)

    # [] means a directed cycle exists.
    return order if len(order) == n else []
```

Data Structures

DSU / Union–Find

1-indexed, iterative path compression, union by size.

```
n = 10
parent = list(range(n + 1))
sz = [1] * (n + 1)

def find(u):
    root = u
    while parent[root] != root:
        root = parent[root]
    while u != root:
        nxt = parent[u]
        parent[u] = root
        u = nxt
    return root

def union(u, v):
    u, v = find(u), find(v)
    if u == v:
        return False
    if sz[u] < sz[v]:
        u, v = v, u
    parent[v] = u
    sz[u] += sz[v]
    return True

# Same component: find(a) == find(b)
# Components after all unions:
# len({find(i) for i in range(1, n + 1)})
```

Fenwick Tree / Binary Indexed Tree

1-based indices $1 \dots N$. Build $O(N)$; point update and prefix/range query $O(\log N)$; space $O(N)$. **Three interpretations:** values by position, frequencies by value, or active flags by position.

Complete SUM Fenwick Code

```
N = 200005
bit = [0] * (N + 1)
arr = [0] * (N + 1)

# Build from a 0-indexed Python list; O(N).
# Resets the old BIT (don't reinsert values).
def build(values):
    if len(values) > N:
        raise ValueError("Too many values")
    bit[:] = [0] * (N + 1)
    arr[:] = [0] * (N + 1)
    for i, x in enumerate(values, 1):
        arr[i] = x
        bit[i] = x
    for i in range(1, N + 1):
        j = i + (i & -i)
        if j <= N:
            bit[j] += bit[i]

# Add val at 1-based index i; NOT replace.
def update(i, val):
    arr[i] += val
    while i <= N:
        bit[i] += val
        i += i & -i

# Inclusive prefix sum [1, i].
def query(i):
    ans = 0
    while i > 0:
        ans += bit[i]
        i -= i & -i
    return ans

# Inclusive sum over [l, r].
def range_query(l, r):
    return query(r) - query(l - 1)

# Replace value at i (not add).
def set_value(i, val):
    update(i, val - arr[i])

# First index i with query(i) >= k.
# Values must be non-negative for kth().
def kth(k):
    if k <= 0 or k > query(N):
        return -1
    i = 0
    step = 1 << (N.bit_length() - 1)
    while step:
        nxt = i + step
        if nxt <= N and bit[nxt] < k:
            k -= bit[nxt]
            i = nxt
        step >>= 1
    return i + 1
```

What Each Operation Means

`build(values)`: reads an ordinary 0-indexed Python list and stores its elements at BIT indices $1, 2, \dots$. Resets previous state.

`update(i,x)`: adds x to position i. For frequency mode, $x = 1$ inserts one occurrence; $x = -1$ removes one occurrence.

`query(i)`: sum from position 1 through i. With frequencies, it counts occurrences with values $\leq i$.

`range_query(l,r)`: `query(r)-query(l-1)`; inclusive ends. In frequency mode, counts values in $[l, r]$, not array indices.

`set_value(i,x)`: replaces the stored value, using `update(i,x-arr[i])`. Never modify `arr[i]` directly.

`kth(k)`: first BIT index whose prefix sum reaches k. In frequency mode it is the k-th smallest value; in active-position mode it is the k-th remaining original position. Returns -1 when invalid.

Quick Start (Fresh Tree)

```
build([2, 4, 1, 5, 3])
query(3)           # 7
range_query(2, 4)  # 10
update(3, 5)       # a[3]: 1 -> 6
range_query(2, 4)  # 15
set_value(2, 10)   # a[2]: 4 -> 10
query(3)           # 18
```

Important Constraints

- Valid update indices: $1 \leq i \leq N$; **never call** `update(0,x)`.
- `kth(k)` requires **non-negative** stored values and $1 \leq k \leq$ `query(N)`.
- Deleting an absent value makes its frequency negative: do not do this.
- A new independent example needs `build([])` (empty BIT) or `build(values)` (fresh initial state).
- Value-indexed frequencies require positive integer values $\leq N$; compress large or negative coordinates.

SUM → XOR Customization

Replace the aggregate operations; the loops stay the same.

Function	Replacement
`build`	`bit[j] ^= bit[i]`
`update`	`arr[i] ^= val; bit[i] ^= val`
`query`	`ans ^= bit[i]`
`range_query`	`query(r) ^ query(l-1)`
`set_value`	`update(i,val ^ arr[i])`

XOR note: `kth()` is **not valid** in XOR mode.

Fenwick Tree — Worked Problem Patterns

Read the representation before choosing a function. Every block below assumes the SUM Fenwick code on the previous page. Examples are independent; `build()` resets the tree.

1. Dynamic Range Sum (by Position)

Problem: Given an array, handle point additions, point assignments, and sum queries over inclusive indices $[l, r]$.
Representation: BIT index = original *position*; BIT value = number stored there.

```
a = [2, 4, 1, 5, 3]
build(a)                # 1-based BIT
query(3)                # 7
range_query(2, 4)       # 10
update(3, 5)            # add 5 to position 3
range_query(2, 4)       # 15
set_value(2, 10)        # replace position 2
query(3)                # 18
```

Choose: `update(i,x)` to add; `set_value(i,x)` to replace; `range_query(l,r)` for sum. Build $O(N)$, then each operation $O(\log N)$.

2. K-th Smallest in a Multiset

Problem: Insert/delete integer values (duplicates allowed), find the k-th smallest, or count elements $\leq x$.
Representation: BIT index = *number value*; BIT value = its *frequency*. Duplicates count separately.

```
build([])                # empty multiset
for x in [5, 2, 8, 2, 6]:
    update(x, 1)         # insert each value
# sorted multiset: [2, 2, 5, 6, 8]
kth(3)                   # 5
query(5)                 # 3 values <= 5
update(2, -1)            # delete ONE 2
# now: [2, 5, 6, 8]
kth(3)                   # 6
```

Requirements: Frequencies must remain non-negative. Values must be integers in $[1, N]$; coordinate compression is needed for very large, negative, or arbitrary integer values. $O(\log N)$ per operation.

Why `kth(k)` Works

For a frequency BIT, `query(i)` counts all elements $\leq i$. So `kth(k)` returns the smallest i with `query(i) >= k`. For active flags, that same i is the original position of the k-th still-present element.

3. K-th Remaining Element (by Position)

Problem: An unsorted array is given with removal queries. Each query k removes and prints the k-th *remaining* element in original order (CSES List Removals). **Representation:** BIT index = *original position*; BIT value = 1 if still active, 0 if removed.

```
a = [2, 6, 1, 4, 2]
removals = [3, 1, 3, 1, 1]
build([1] * len(a))       # all positions active
ans = []
for k in removals:
    pos = kth(k)          # original 1-based index
    ans.append(a[pos-1])  # read original value
    update(pos, -1)       # active 1 -> 0
print(*ans)               # 1 2 2 6 4
```

Key detail: `a` never changes; `update(pos,-1)` only marks that position as removed. `a[pos-1]` converts the BIT's 1-based position to Python's 0-based index. Each removal $O(\log N)$. No coordinate compression is needed, even when array values are huge or negative.

4. Count Values in a Range

Problem: Maintain a collection with insertions, deletions, and queries asking how many values are in $[l, r]$ inclusive, *not* how many array positions lie in that interval.
Representation: BIT index = *number value*; BIT value = its frequency.

```
build([])                # empty frequency BIT
for x in [2, 2, 5, 6, 8]:
    update(x, 1)
range_query(2, 5)        # 3 (2, 2, 5)
range_query(6, 8)        # 2 (6, 8)
update(5, 1)             # insert another 5
range_query(2, 5)        # 4
```

Reason: `range_query(l,r)` equals `query(r)-query(l-1)`; for a frequency BIT this counts values, including duplicates, in the range. Each insertion/deletion/query $O(\log N)$. Value range must fit $[1, N]$ or be compressed first.

Choose the Correct Representation

Problem	BIT stores
Dynamic range sum	Values at positions
K-th smallest	Frequencies by value
K-th remaining	Active flags by position
Count values in range	Frequencies by value

Segment Tree — Basic (Iterative)

0-based indexing, inclusive $[l, r]$**.** Build $O(n)$, point update and range query $O(\log n)$, space $O(n)$.

Segment Tree — Basic (Iterative)

```
from math import gcd

# Choose before building (default: MIN)
combine = min
IDENTITY = float('inf')

# Build from a 0-indexed list, O(n)
def seg_build(arr):
    n = len(arr)
    size = 1
    while size < n:
        size <<= 1
    tree = [IDENTITY] * (2 * size)
    tree[size:size + n] = arr
    for i in range(size - 1, 0, -1):
        tree[i] = combine(tree[2*i],
                          tree[2*i+1])
    return tree, size

# Inclusive query [l, r]
def seg_query(tree, size, l, r):
    l += size
    r += size
    left = right = IDENTITY
    while l <= r:
        if l & 1:
            left = combine(left, tree[l])
            l += 1
        if not (r & 1):
            right = combine(tree[r], right)
            r -= 1
        l >>= 1
        r >>= 1
    return combine(left, right)

# Replace arr[i] with val
def seg_update(tree, size, i, val):
    i += size
    tree[i] = val
    i >>= 1
    while i:
        tree[i] = combine(tree[2*i],
                          tree[2*i+1])
        i >>= 1

# Add val to arr[i] (point addition)
def seg_add(tree, size, i, val):
    old = tree[size + i]
    seg_update(tree, size, i, old + val)
```

Initialize

```
a = [8, 3, 5, 1, 6]
tree, size = seg_build(a)
seg_query(tree, size, 1, 4)  # 1
seg_update(tree, size, 3, 9)
seg_query(tree, size, 1, 4)  # 3
seg_add(tree, size, 1, -2)
seg_query(tree, size, 1, 4)  # 1
```

Customize: `combine` + `IDENTITY`

Change **both** lines *before* `seg_build(arr)`. The rest of the template stays the same.

Query	combine	IDENTITY
SUM	`lambda a,b:a+b`	`0`
MIN	`min`	`float('inf')`
MAX	`max`	`float('-inf')`
XOR	`lambda a,b:a^b`	`0`
GCD	`gcd`	`0`
AND	`lambda a,b:a&b`	`-1`
OR	`lambda a,b:a|b`	`0`

Recognize These Problem Patterns

1. Dynamic Range Sum
Problem: Point assignments/additions; sum of positions $[l, r]$.
Approach: Use SUM mode, `seg_update`, `seg_add`, `seg_query`.

2. Range Minimum / Maximum
Problem: Update individual values; find smallest/largest in $[l, r]$.
Approach: Use MIN or MAX mode.

3. Dynamic Range GCD
Problem: Update one value; calculate GCD over $[l, r]$.
Approach: Use `combine = gcd`, `IDENTITY = 0`.

4. Range XOR
Problem: Point updates and XOR over $[l, r]$.
Approach: Use XOR mode; aggregate neighboring segments with XOR.

5. Bitwise AND / OR
Problem: Point updates and bitwise AND/OR over $[l, r]$.
Approach: Use AND (identity -1) or OR (identity 0).

Quick Usage: SUM Mode

```
combine = lambda a, b: a + b
IDENTITY = 0
tree, size = seg_build([2, 4, 1, 5, 3])
seg_query(tree, size, 1, 3)  # 10
seg_update(tree, size, 2, 7)
seg_query(tree, size, 1, 3)  # 16
seg_add(tree, size, 1, 3)
seg_query(tree, size, 1, 3)  # 19
```

Important Notes

- Queries are **inclusive** $[l, r]$; indices start at 0. Use a non-empty array and valid indices.
- `seg_update(i,x)` **replaces**; `seg_add(i,x)` **adds**, regardless of query mode. Pass `tree,size` first.
- The combine operation must be **associative**; identity must be neutral on both sides.
- Two query accumulators preserve order for noncommutative operations.
- Rebuild the tree after changing combine/identity. For range updates use a **Lazy Segment Tree**.

Basic Segment Tree — Worked Problem Patterns

Always choose combine and IDENTITY before seg_build(). Each example below is independent and assumes the basic Segment Tree functions on the preceding page. External indices are 0-based; $[l, r]$ is inclusive.

1. Dynamic Range Sum

Problem: Assign/add at one index and query sums over $[l, r]$. Each tree node stores its segment sum. Use SUM configuration.

```
combine = lambda a, b: a + b
IDENTITY = 0
tree, size = seg_build([2, 4, 1, 5, 3])
seg_query(tree, size, 1, 3)   # 10
seg_update(tree, size, 2, 7) # set a[2]=7
seg_query(tree, size, 1, 3)   # 16
seg_add(tree, size, 1, 3)    # add 3 to a[1]
seg_query(tree, size, 1, 3)   # 19
```

Typical problem: CSES Dynamic Range Sum Queries. Updates and queries cost $O(\log n)$.

2. Dynamic Range Minimum / Maximum

Problem: Replace/add at one index; ask for the smallest/largest value in a subarray. Set `min/+inf` or `max/-inf` *before building.*

```
combine = min
IDENTITY = float('inf')
tree, size = seg_build([8, 3, 5, 1, 6])
seg_query(tree, size, 1, 4)   # 1
seg_update(tree, size, 3, 9) # a[3]: 1 -> 9
seg_query(tree, size, 1, 4)   # 3
seg_add(tree, size, 1, -2)   # a[1]: 3 -> 1
seg_query(tree, size, 1, 4)   # 1
```

Typical problem: CSES Dynamic Range Minimum Queries.

3. Dynamic Range GCD

Problem: Point assignment and GCD over $[l, r]$. GCD is associative and `gcd(x,0)=x`.

```
from math import gcd
combine = gcd
IDENTITY = 0
tree, size = seg_build([12, 18, 6, 15])
seg_query(tree, size, 0, 2)   # 6
seg_update(tree, size, 2, 9)
seg_query(tree, size, 0, 2)   # 3
```

4. Dynamic Range XOR

Problem: Point assignment, XOR across $[l, r]$. XOR identity is 0; changing one leaf recalculates its ancestors.

```
combine = lambda a, b: a ^ b
IDENTITY = 0
tree, size = seg_build([1, 2, 3, 4])
seg_query(tree, size, 0, 2)   # 0
seg_update(tree, size, 2, 7)
seg_query(tree, size, 0, 2)   # 4
```

5. Range Bitwise AND / OR

Problem: Update a single value, answer AND/OR over $[l, r]$. `-1` is the all-ones identity for Python integer AND; `0` is the identity for OR.

```
a = [6, 3, 5]
combine = lambda a, b: a & b
IDENTITY = -1
tree, size = seg_build(a)
seg_query(tree, size, 0, 2)  # 0

combine = lambda a, b: a | b
IDENTITY = 0
tree, size = seg_build(a)     # rebuild!
seg_query(tree, size, 0, 2)  # 7
```

What Each Function Does

`seg_build(a)`: Fill power-of-two-sized leaves with the original array and pad with the identity; compute parents; return `tree,size`. $O(n)$.

`seg_query(tree,size,l,r)`: Combine nodes covering inclusive $[l, r]$. Two accumulators preserve operation order. $O(\log n)$.

`seg_update(tree,size,i,x)`: *replace* `a[i]` by x; recalculate its ancestors. $O(\log n)$.

`seg_add(tree,size,i,x)`: *add* x to the value at i, even in MIN/MAX/GCD/XOR modes; recalculate. $O(\log n)$.

Contest Safety Checklist

- Valid array is non-empty; valid original indices $0 \le l \le r < n$.
- The combine operation must be *associative* and the identity neutral on both sides.
- Changing combine or identity requires rebuilding.
- No $O(\log n)$ range updates here: use Lazy Segment Tree.

Lazy Segment Tree — Iterative

0-based, inclusive $[l, r]$**.** Range add/set + SUM/MIN/MAX queries $O(\log n)$; build $O(n)$, space $O(n)$. No recursion.

```
# SUM / MIN / MAX; 0-based, inclusive [l,r]
MODE = "SUM"

def build(a):
    global n,size,height,tree,lazy_add,lazy_set,seg_len
    global combine,IDENTITY
    n = len(a)
    if not n: raise ValueError("Empty array")
    if MODE == "SUM":
        combine,IDENTITY = (lambda x,y:x+y),0
    elif MODE == "MIN":
        combine,IDENTITY = min,float('inf')
    elif MODE == "MAX":
        combine,IDENTITY = max,float('-inf')
    else: raise ValueError("Invalid MODE")
    size = 1 << (n-1).bit_length()
    height = size.bit_length()-1
    tree = [IDENTITY]*(2*size)
    tree[size:size+n] = a
    lazy_add = [0]*size
    lazy_set = [None]*size
    seg_len = [1]*(2*size)
    for p in range(size-1,0,-1):
        seg_len[p] = seg_len[2*p]+seg_len[2*p+1]
        tree[p] = combine(tree[2*p],tree[2*p+1])

def _set(p,x):
    tree[p] = x*seg_len[p] if MODE=="SUM" else x
    if p < size:
        lazy_set[p],lazy_add[p] = x,0

def _add(p,x):
    tree[p] += x*seg_len[p] if MODE=="SUM" else x
    if p < size:
        if lazy_set[p] is not None: lazy_set[p] += x
        else: lazy_add[p] += x

def _push(p):
    if lazy_set[p] is not None:
        _set(2*p,lazy_set[p]); _set(2*p+1,lazy_set[p])
        lazy_set[p] = None
    if lazy_add[p]:
        _add(2*p,lazy_add[p]); _add(2*p+1,lazy_add[p])
        lazy_add[p] = 0

def _push_paths(L,R):
    for h in range(height,0,-1):
        if L & ((1<<h)-1): _push(L>>h)
        if R & ((1<<h)-1): _push((R-1)>>h)

def _pull_paths(L,R):
    for h in range(1,height+1):
        if L & ((1<<h)-1):
            p = L>>h
            tree[p] = combine(tree[2*p],tree[2*p+1])
        if R & ((1<<h)-1):
            p = (R-1)>>h
            tree[p] = combine(tree[2*p],tree[2*p+1])

def _apply(l,r,x,fn):
    L,R = l+size,r+size+1
    _push_paths(L,R)
    a,b = L,R
    while a < b:
        if a&1: fn(a,x); a += 1
        if b&1: b -= 1; fn(b,x)
        a >>= 1; b >>= 1
    _pull_paths(L,R)

def range_add(l,r,x): _apply(l,r,x,_add)
def range_set(l,r,x): _apply(l,r,x,_set)
def add_value(i,x): range_add(i,i,x)
def set_value(i,x): range_set(i,i,x)

def query(l,r):
    L,R = l+size,r+size+1
    _push_paths(L,R)
    left = right = IDENTITY
    while L < R:
        if L&1: left = combine(left,tree[L]); L += 1
        if R&1: R -= 1; right = combine(tree[R],right)
        L >>= 1; R >>= 1
    return combine(left,right)
```

Configuration

Set `MODE` to `"SUM"`, `"MIN"`, or `"MAX"` before `build(a)`. Rebuilding resets all lazy tags.

Mode	Combine	Neutral
SUM	`x+y`	0
MIN	`min(x,y)`	$+\infty$
MAX	`max(x,y)`	$-\infty$

Update Rules

Mode	Add x	Set to x
SUM	`tree[p] += x * seg_len[p]`	`x * seg_len[p]`
MIN/MAX	`tree[p] += x`	`x`

Priority: SET overrides old additions. ADD after SET increases the pending set value. `None` means no pending SET; zero is valid.

Recognize These Problem Patterns

1. Range Add + SUM
Problem: Add x to every element in $[l, r]$; query range sums.
Approach: SUM mode; `range_add(l,r,x)`, `query(l,r)`.

2. Range Set + SUM
Problem: Assign all values in $[l, r]$ to x; query sums.
Approach: SUM mode; `range_set(l,r,x)`.

3. Range Add/Set + MIN/MAX
Problem: Apply updates to intervals; find minimum/maximum.
Approach: Use MIN or MAX mode; same update/query functions.

4. Overlapping Updates
Problem: Several range additions and assignments overlap.
Approach: Lazy tags preserve operation order; later SET overwrites old updates, later ADD increments pending SET.

Quick Usage (SUM)

```
MODE = "SUM"
build([1, 2, 3, 4])
range_add(0, 3, 5)  # [6, 7, 8, 9]
range_set(1, 2, 0)  # [6, 0, 0, 9]
range_add(2, 3, 2)  # [6, 0, 2, 11]
query(0, 3)        # 19
set_value(0, 4)    # replace a[0]
add_value(1, 3)    # add 3 to a[1]
```

Important Notes

- Valid range: $0 \le l \le r < n$; endpoints inclusive.
- Internally use half-open $[L, R)$ with `R=r+size+1`.
- Push boundary paths *before* applying/querying; pull affected ancestors *after* updating.
- Range ADD / SET both $O(\log n)$; setting a point is just a length-one range update.
- Choose MODE before build; do not change it mid-operation.

Lazy Segment Tree — Mechanics and Applications

Choose MODE = SUM/MIN/MAX before build(). 0-based inclusive $[l, r]$. The core on the preceding page supports $O(\log n)$ range ADD, SET, and queries. The examples below use fresh builds.

How Lazy Propagation Works

`tree[p]` stores the result for the whole segment at node p. `seg_len[p]` stores that segment's length. `lazy_add[p]` is an addition postponed for children; `lazy_set[p]` is a postponed assignment (`None` means no pending SET, but 0 is a valid SET).

`_set(p,x)`: A whole segment becomes x. SUM node becomes $x \times$ length; MIN/MAX node becomes x. Forget older pending ADD and remember the new SET.

`_add(p,x)`: Increase an entire segment by x. SUM increases by $x \times$ length; MIN/MAX by x. If SET was pending, add x to that SET value; otherwise accumulate ADD.

`_push(p)`: Pass pending SET to both children first, then pending ADD; reset the tags on p. The children are now safe to inspect or modify.

`_push_paths(L,R)`: Push delayed changes down the two split boundary paths before a query/update. `_pull_paths(L,R)`: After the update, recompute affected boundary ancestors without overwriting a fully-covered node that already has its correct lazy aggregate.

`_apply(l,r,x,fn)`: Convert inclusive $[l, r]$ to internal half-open $[l + size, r + size + 1)$; push boundary paths, update covered nodes, then pull boundary ancestors.

Order of Overlapping Updates

```
MODE = "SUM"
build([1, 2, 3, 4])
range_add(0, 3, 5)  # [6, 7, 8, 9]
range_set(1, 2, 0)  # [6, 0, 0, 9]
range_add(2, 3, 2)  # [6, 0, 2, 11]
query(0, 3)        # 19
```

Rule: Later SET overrides previous additions in that interval. Later ADD increases the pending SET. That is why the order of operations matters.

Public Function Summary

`build(a)` resets all arrays and tags; `range_add(l,r,x)` adds x to every value; `range_set(l,r,x)` replaces every value by x; `query(l,r)` returns SUM/MIN/MAX; `add_value(i,x)` and `set_value(i,x)` are point versions.

1. Range ADD + Range SUM

Problem: Add x to each element in $[l, r]$ and ask for the sum of a range. SUM mode; whole-segment sum changes by $x \times$ length.

```
MODE = "SUM"
build([2, 4, 1, 5, 3])
range_add(1, 3, 10)
# [2, 14, 11, 15, 3]
query(1, 3)  # 40
query(0, 4)  # 45
```

2. Range SET + Range SUM

Problem: Replace every value in $[l, r]$ with x; answer sum queries. A segment of length k set to x sums to kx.

```
MODE = "SUM"
build([2, 4, 1, 5, 3])
range_set(1, 3, 7)
# [2, 7, 7, 7, 3]
query(0, 4)  # 26
```

3. Range ADD / SET + MIN / MAX

Problem: Interval updates and min/max queries. In MIN/MAX mode ADD increases that aggregate by x; SET changes that aggregate to x.

```
MODE = "MIN"  # alternatively "MAX"
build([8, 3, 5, 1, 6])
range_add(1, 3, 4)
# [8, 7, 9, 5, 6]
query(0, 4)  # MIN: 5; MAX: 9
range_set(2, 4, 2)
# [8, 7, 2, 2, 2]
query(0, 4)  # MIN: 2; MAX: 8
```

For the MAX answers, change MODE to `"MAX"` and call `build()` again before both updates.

4. Overlapping ADD and SET

Problem: Range assignments/additions overlap. The same ADD/SET functions preserve chronological order of updates. See the fully traced example at left.

Important Limits

- Only SUM/MIN/MAX modes and arithmetic ADD/SET are supported; other operations require new lazy rules.
- External indices must satisfy $0 \le l \le r < n$. Internally $R = r + size + 1$.
- Changing MODE requires rebuilding.
- All operations above $O(\log n)$, build and memory $O(n)$.
- Avoid treating `lazy_set[p] == 0` as no pending assignment.

Segment Tree Search — Iterative

0-based indexing. Find first $\geq x$, k-th active, or first prefix sum $\geq x$. Build $O(n)$, search/update/query $O(\log n)$, space $O(n)$.

```
# Set MODE before building: SUM / MAX
MODE = "SUM"

def build(a):
    global n,size,tree,combine,IDENTITY
    n = len(a)
    if not n: raise ValueError("Empty array")
    if MODE == "SUM":
        combine,IDENTITY = (lambda x,y:x+y),0
    elif MODE == "MAX":
        combine,IDENTITY = max,float('-inf')
    else: raise ValueError("Invalid MODE")
    size = 1 << (n-1).bit_length()
    tree = [IDENTITY]*(2*size)
    tree[size:size+n] = a
    for p in range(size-1,0,-1):
        tree[p] = combine(tree[2*p],tree[2*p+1])

# Point replacement and addition
def set_value(i,x):
    p = i+size
    tree[p] = x
    while p > 1:
        p >>= 1
        tree[p] = combine(tree[2*p],tree[2*p+1])

def add_value(i,x):
    set_value(i,tree[size+i]+x)

# Inclusive [l,r]: range SUM or MAX
def query(l,r):
    l += size; r += size
    left = right = IDENTITY
    while l <= r:
        if l&1:
            left = combine(left,tree[l]); l += 1
        if not (r&1):
            right = combine(tree[r],right); r -= 1
        l >>= 1; r >>= 1
    return combine(left,right)

# MAX mode: first i >= start with a[i] >= x
# Unsorted array is fine; -1 if absent
def first_at_least(x,start=0):
    if MODE != "MAX": raise ValueError("MAX only")
    if start >= n: return -1
    p = start+size
    while True:
        while p%2 == 0: p >>= 1
        if tree[p] >= x:
            while p < size:
                p *= 2
                if tree[p] < x: p += 1
            i = p-size
            return i if i < n else -1
        p += 1
        if (p & (p-1)) == 0: break
    return -1

# SUM mode: first prefix sum >= target
# All values NON-NEGATIVE; target > 0
def prefix_lower_bound(target):
    if MODE != "SUM": raise ValueError("SUM only")
    if target <= 0 or target > tree[1]:
        return -1
    p = 1
    while p < size:
        p *= 2
        if tree[p] < target:
            target -= tree[p]; p += 1
    i = p-size
    return i if i < n else -1

# SUM mode: 1-based k, returns 0-based index
# Requires non-negative integer frequencies
def kth(k):
    return prefix_lower_bound(k)
```

Configuration and Searches

Choose `MODE` *before* `build(a)`.

Mode	Node stores	Use for
MAX	Maximum	`first_at_least`
SUM	Sum	`kth`, `prefix_lower_bound`

Each search descends through the tree in $O(\log n)$; no repeated binary-search range queries.

Recognize These Problem Patterns

1. First Element $\geq x$
Problem: Update an unsorted array; find the first index $i \geq start$ with `a[i] >= x`.
Approach: MAX mode. Skip nodes whose maximum is $< x$; descend to the leftmost qualifying leaf.

2. k-th Remaining Element
Problem: Repeatedly remove the k-th still-active item and report its original value.
Approach: SUM mode. Build a tree of ones. Find `pos=kth(k)`, read `a[pos]`, then `set_value(pos,0)`.

3. Prefix Sum Lower Bound
Problem: Find first i with $a_0 + \cdots + a_i \geq x$.
Approach: SUM mode with non-negative values. Go left if the left sum reaches x; otherwise subtract it and go right.

4. k-th Smallest in a Multiset
Problem: Insert/delete values with duplicates; find the k-th.
Approach: SUM mode: leaves represent *values*, and store frequencies. Use `add_value(x,1/-1)` and `kth(k)`.

Quick Usage Examples

```
# First >= 6 (MAX mode):
MODE = "MAX"
build([2, 4, 1, 8, 3, 7])
first_at_least(6)       # 3
first_at_least(6, 4)    # 5

# K-th active (SUM mode, new build):
MODE = "SUM"
build([1, 0, 1, 1, 0, 1])
kth(3)                  # 3
set_value(3, 0)         # deactivate
kth(3)                  # 5

# Prefix lower bound (new build):
build([2, 0, 3, 1])
prefix_lower_bound(5)   # 2
```

Important Notes

- Results are **0-based indices**; -1 means not found.
- `kth(k)` takes **1-based** k.
- SUM descent requires **non-negative** values; for k-th queries, use non-negative integer frequencies.
- `prefix_lower_bound(x)` expects $x > 0$; this template returns -1 for $x \leq 0$.
- MAX search does not require sorting. Never delete an absent value; compress large coordinates.
- Changing MODE requires rebuilding; only point updates.

Segment Tree Search — Worked Problem Patterns

Change MODE before build(). Each example below starts with a fresh build. External indices are 0-based. Searches $O(\log n)$. The code on the preceding page performs descent within the tree, not binary search with repeated range queries.

1. First Element $\geq x$ (MAX Mode)

Problem: On an *unsorted* array with point updates, find the first original index $i \geq start$ for which `a[i]>=x`. If a node's maximum is $< x$, skip its whole segment; otherwise descend to the leftmost valid leaf.

```
MODE = "MAX"
build([2, 4, 1, 8, 3, 7])
first_at_least(6)       # 3
first_at_least(6, 4)    # 5
set_value(3, 2)
first_at_least(6)       # 5
```

Important: No sorting needed. Searches are restricted to original indices $i \geq start$. Returns -1 if absent.

2. k-th Remaining Element (SUM Mode)

Problem: Repeatedly remove the k-th remaining element, printing original array values in original order. Build *active flags*: 1 for present, 0 for removed. `kth(k)` returns the *original 0-based position*.

```
MODE = "SUM"
a = [2, 6, 1, 4, 2]
removals = [3, 1, 3, 1, 1]
build([1] * len(a))
ans = []
for k in removals:
    pos = kth(k)
    ans.append(a[pos])
    set_value(pos, 0)
print(ans)  # [1, 2, 2, 6, 4]
```

Difference from Fenwick: This segment tree returns 0-based positions; the Fenwick `kth(k)` returns a 1-based BIT index. No removal from the actual `a` list.

How the Two Modes Differ

MAX stores segment maxima and supports `first_at_least(x,start)`. **SUM** stores non-negative sums/frequencies and supports `prefix_lower_bound(target)` and `kth(k)`. Changing MODE requires another build.

3. First Prefix Sum $\geq x$ (SUM Mode)

Problem: Find the first index i with $a_0 + \cdots + a_i \geq x$. All values must be *non-negative* so prefix sums are nondecreasing. To descend: if the left child's sum reaches the target, go left; otherwise subtract that sum and go right.

```
MODE = "SUM"
build([2, 0, 3, 1])
# Prefix sums: [2, 2, 5, 6]
prefix_lower_bound(1)  # 0
prefix_lower_bound(3)  # 2
prefix_lower_bound(5)  # 2
prefix_lower_bound(6)  # 3
prefix_lower_bound(7)  # -1
```

This implementation returns -1 when `target<=0` or the target is beyond the total sum.

4. k-th Smallest in a Multiset (SUM Mode)

Problem: Insert/delete integer values (duplicates allowed) and find the k-th smallest occurrence. Here leaf index = *value*; leaf value = *frequency*. Therefore `kth(k)` gives a value index, not a position in the original input list.

```
MODE = "SUM"
# Allowed values: 0,1,2,3,4
build([0, 2, 1, 0, 3])
# Values: [1, 1, 2, 4, 4, 4]
kth(1)  # 1
kth(3)  # 2
kth(4)  # 4
add_value(2, -1)  # remove one 2
kth(3)  # 4
```

Requirements: Non-negative integer frequencies; never remove an absent value. Large/negative value domains may need coordinate compression and mapping back to real values.

Search Safety Notes

- k is 1-based, returned index is 0-based; -1 means invalid/not found.
- Prefix lower bound requires all numbers ≥ 0; k-th requires non-negative integer counts.
- Only point updates: use Lazy Segment Tree for interval updates.
- `query(l,r)` is SUM or MAX over inclusive indices.